VOYQI MOBILITY OS

System Architecture — Implementation Guide

"Your Mobility Travels With You™"

Revision E · 26 Microservices · 12-Agent AI Orchestration Nucleus · Google Cloud Platform

Prepared for the VOYQI Holdings Inc. engineering team

This document reconciles and supersedes architecture notes in earlier drafts of the PRD, Engineer Handoff, and prior build sessions. Where anything elsewhere conflicts with this guide on cloud provider, service count, agent roster, or autonomy tiering, this guide is correct.

1. Purpose & How to Use This Guide

This is the implementation guide for the VOYQI Mobility OS system architecture — the document that takes an engineer from zero to a running, correctly-configured platform. It consolidates the architecture, infrastructure, AI orchestration design, and autonomy model into one sequence: what to build, in what order, and why each decision was made.

Three companion documents live alongside the infrastructure repository and are referenced throughout rather than duplicated in full: PORT_REGISTRY.md (authoritative port list), SERVICE_INVENTORY.md (per-service build status), and AUTONOMY_TIERING_ENGINEER_INSTRUCTIONS.md (the module-by-module autonomy rollout, including the Fleet Management diff). This guide is the map; those are the detailed references.

> **Status:** Sections 2–7 describe the target-state architecture, most of which is provisioned in the infrastructure repository already. Section 8 (autonomy tiering) is specified but not yet applied to the live server — see the Rollout heading in Section 8 for what's pending.

2. Executive Summary

VOYQI Mobility OS is an AI-native "Portable Mobility™" platform: members subscribe to a vehicle class that travels with them across hub cities. The platform is built as 26 microservices across three architecture layers, orchestrated by a 12-agent AI nucleus, deployed on Google Cloud Platform.

- Layer 0 — Platform Infrastructure: 2 services (gateway, auth)
- Layer 1 — Core Execution: 12 services (11 execution services + the AI orchestration nucleus, itself split across 2 deployment tiers)
- Layer 2 — Admin-Panel-Backing: 12 services, backing the 17 primary panels of the Enterprise Admin Screen Specification

The defining architectural decision is that AI is a system layer, not a bolt-on feature: the orchestration nucleus decides, execution services persist and validate, and every AI judgment has a deterministic fallback so the platform is never dependent on a model call for a critical path.

3. Architectural Principles

These are enforced, not aspirational — each has a concrete mechanism cited alongside it.

- AI is the system layer. The nucleus decides; execution services persist, validate, and execute.
- "AI proposes, human approves" is replaced by a 3-tier autonomy model (Section 8) — the blanket rule made the platform a recommendation engine, not an automated one.
- Two-layer agent discipline: every AI judgment has a deterministic fallback mirroring backend logic exactly.
- Structural separation of AI-judged vs. human-only fields: liability, collateral status, and tamper events never enter the AI decision path, at any autonomy tier.
- API-first, gateway-only ingress: no service port is ever reachable directly by a client — enforced by Kubernetes NetworkPolicy.
- Database-per-service: no shared databases, no cross-service SQL joins.
- Multi-tenant by construction: every request carries X-Tenant-Id; every table is tenant-scoped.
- JWT via HttpOnly cookies only — never localStorage.
- Every state change publishes a Kafka event; the AI nucleus consumes all of it.
- Cloud-native GCP (GKE + Cloud SQL) — not AWS. An earlier draft targeted AWS; this guide and the infrastructure repository are the corrected source.

4. System Architecture

Client traffic (Member App on Flutter, Operator Portal and Executive Console on React/TanStack Start) enters exclusively through the API Gateway on port 8000, which routes to either the AI orchestration nucleus or the 25 execution/admin services. Every service publishes to Kafka; the AI nucleus consumes every topic. All persistent state lives in Cloud SQL (two instances — one for Layer 0/1, one for Layer 2), Redis (Memorystore), and Mosquitto/EMQX for vehicle telematics over MQTT.

4.1 Responsibility Split

Layer	Owns	Does not do
AI Orchestration	Decisions, context, anticipation, optimization, goals	Store data, handle HTTP, validate inputs
Execution Services (1/2)	Persistence, input validation, instruction execution, event publishing	Autonomous decisions, business judgment
API Gateway	Authentication, routing, rate limiting	Decisions, data storage
Frontend	Member/operator experience, rendering AI decisions	Business logic

5. Technology Stack

Layer	Technology	Notes
Backend services	Python, FastAPI, SQLAlchemy (async)	26 services, identical structural pattern: models / services / api / core / tests
Database	PostgreSQL 16 (Cloud SQL)	Database-per-service; SQLite + StaticPool for test isolation
AI orchestration	Anthropic Claude + LangGraph	Supervisor + 11 specialist agents across 2 scheduling tiers (Section 8)
Event streaming	Apache Kafka (Strimzi, self-hosted on GKE)	Split consumer groups per AI tier
Frontend (web)	TypeScript, React, TanStack Start, Tailwind	Member App, Operator Portal, Executive Console
Frontend (mobile)	Flutter	Riverpod + GoRouter
Cloud infrastructure	GCP — GKE + Cloud SQL	Corrected from an earlier AWS draft
Cache / sessions / checkpoint	Redis (Memorystore)	Rate limits, pricing cache, and the externalized LangGraph checkpoint store
IoT / telematics	Eclipse Mosquitto (Ph.1) → EMQX (Ph.2)	MQTT over TLS, client cert CN = VIN
IaC	Terraform (GCP provider) + Kustomize	infrastructure/ in the repository

6. Infrastructure Implementation

6.1 Networking

One VPC per environment (dev/staging/prod), private GKE nodes, Cloud NAT for outbound-only egress, and a reserved private-services range peered for Cloud SQL — no Cloud SQL instance ever has a public IP.

6.2 GKE — Three Node Pools

The AI orchestration nucleus is scheduled across two dedicated pools rather than sharing one, because CPU contention happens at the node level: a batch reasoning job cannot be allowed to starve a member-facing chat reply even if they're in separate pods, unless they're also on separate nodes.

Pool	Workload	QoS / Priority	Scaling (pr
general-pool	Gateway, Layer 0/1/2 execution & admin services	Standard	3–12 nodes,
ai-realtime-pool	AI orchestration — realtime tier	Guaranteed QoS, PriorityClass 900000	2–8 nodes, r
ai-orchestration-batch-pool	AI orchestration — batch tier	Burstable QoS, PriorityClass 100000	2–6 nodes, r

6.3 Cloud SQL

Two Postgres 16 instances: one hosting the 13 Layer 0/1 service databases (including the single shared ai_orchestration_service database — see Section 8.1 on why this stays one database despite two deployment tiers), one hosting the 12 Layer 2 databases. Each service gets its own database and a least-privilege user with zero cross-database grants.

6.4 Kafka

Self-hosted via the Strimzi operator (KRaft mode, 3 brokers in prod) rather than a managed GCP substitute, so existing Kafka client code needs no changes for GCP. The AI nucleus's consumer group is split in two — ai-orchestration-realtime and ai-orchestration-batch — each subscribed only to its own topic set, so a batch-side backlog can never delay realtime event processing.

6.5 Redis

Memorystore, STANDARD_HA tier in production. Two roles: rate-limit counters / pricing cache, and the externalized LangGraph checkpoint store keyed by thread_id, so any AI replica in either tier can service any conversation turn — no sticky sessions, no state lost on pod restart.

6.6 IAM

Four Google Service Accounts (gateway, core services, AI orchestration, admin services) bound to Kubernetes Service Accounts via Workload Identity — no downloaded JSON keys anywhere in the platform.

7. Service Catalog

Single source of truth: infrastructure/kubernetes/services-catalog.yaml. api-gateway, ai-orchestration-realtime, and ai-orchestration-batch are hand-authored; the other 24 services are generated from one cloneable template via scripts/generate_service_manifests.py, so they stay structurally identical.

7.1 Layer 0 — Platform Infrastructure

Service	Port	Status
api-gateway	8000	READY
auth-service	8001	READY

7.2 Layer 1 — Core Execution & AI Nucleus

Service	Port	Status
user-service	8002	READY

Service	Port	Status
subscription-service	8003	READY
fleet-service	8004	READY
rental-service	8005	READY
payment-service	8006	READY
marketing-service	8007	READY
sales-service	8009	READY
pricing-service	8010	READY
rbac-service	8011	READY
notification-service	8012	READY
global-mobility-svc	8013	READY
ai-orchestration-batch	8014	READY
ai-orchestration-realtime	8020	READY

7.3 Layer 2 — Admin-Panel-Backing

Service	Port	Status
super-admin-svc	8021	ACTIVE_DEV
crm-svc	8022	PLANNED
dms-svc	8023	PLANNED
originations-svc	8024	PLANNED
executive-svc	8025	PLANNED
management-svc	8026	PLANNED
real-estate-svc	8027	PLANNED
capital-finance-svc	8028	PLANNED
insurance-risk-svc	8029	PLANNED
technology-ip-svc	8030	PLANNED
security-compliance-svc	8031	PLANNED
hrm-svc	8101	ACTIVE DEV

> **hrm-svc port note:** 8101, intentionally outside the 8021–8031 block — a historical allocation carried forward rather than renumbered. Full detail: docs/PORT_REGISTRY.md.

8. AI Orchestration Nucleus

8.1 One Logical Service, Two Scheduling Tiers

The nucleus is one LangGraph graph, one image, one Supervisor — split into two Kubernetes deployment groups purely for scheduling isolation. Both tiers read and write the same Postgres decision log and the same Redis-backed checkpoint store. This is a deliberate design constraint: the platform's own PRD rules out building the AI as multiple independent services, because independent services don't share context.

	Realtime (port 8020)	Batch (port 8014)
Agents	Supervisor, Member Experience, Concierge, Relationship Intelligence	Fleet Intelligence, Pricing Intelligence, Revenue, Retention, Sales & Originations Intelligence, Talent & Workforce Intelligence, Compliance & Risk Intelligence, Portfolio Intelligence
QoS	Guaranteed (requests = limits)	Burstable
Priority	900000	100000
HPA signal	p95 latency (CPU fallback)	CPU utilization
Rationale	A human is waiting in-session	Throughput-heavy or non-interactive reasoning

This split independently fixes four compromises that a single shared deployment would have: node-level resource contention, Anthropic API rate-limit contention (two Workspaces, two keys, each with its own carved-out rate-limit allocation), deploy blast radius (separate Deployments, maxUnavailable: 0, canary-before-promote), and session/checkpoint affinity (externalized to Redis, no sticky sessions).

8.2 The 12-Agent Roster

The original 7-agent design covered only the core Mobility platform. The Enterprise Admin Screen Specification later assumed CRM, Originations, and Talent-CRM agents that were never formally defined — a gap introduced across separate build sessions. Five agents close it, matched to distinct judgment domains rather than one per service:

Agent	Tier	Powers
Relationship Intelligence	Realtime	crm-svc — staff wait on drafted replies in-session
Sales & Originations Intelligence	Batch	sales-service, dms-svc, originations-svc
Talent & Workforce Intelligence	Batch	hrm-svc
Compliance & Risk Intelligence	Batch	security-compliance-svc, insurance-risk-svc
Portfolio Intelligence	Batch	executive-svc, management-svc, real-estate-svc, capital-finance-svc

8.3 Autonomy Tiering — Replacing the Blanket Approval Rule

A blanket "AI proposes, human approves" rule for every action makes the platform a recommendation engine: a human clicks approve on routine, reversible, low-stakes work that never needed a person. This is replaced with a three-tier model, enforced by infrastructure/kubernetes/base/ai-orchestration/tier-policy.yaml — a mounted ConfigMap both AI deployments consult before executing any action, not a UI convention.

Tier	Definition	Mechanism
1 — Fully autonomous	Reversible, low-stakes, high-volume	Executes immediately, logs to the decision log, reviewable after the fact
2 — Approval gate	Low confidence, above a cost/scale threshold	Queues in AI-002 (Recommendation Review & Override), waits for a click
3 — Always human	Regulatorily required accountable decision (fair lending, employment law, access-policy grants)	Hard-blocked — not reachable via any threshold change; requires a named accountable role, not just a click

default_tier is 2 — an action type not explicitly listed requires approval until someone deliberately adds it. Silence must never mean autonomous.

8.4 Fleet Management — Promoted to Full Automation

Fleet is the one module with real production history, and it has been promoted: 11 of its 13 action types are Tier 1. The two exceptions — incident.liability_assessment and telematics.tamper_event_flag — are not a tiering question at all; they are human-only fields structurally excluded from the AI decision path in the data model, and are not eligible for promotion regardless of autonomy policy.

Action	Tier
staging.recommend_prestage	1
staging.rebalance_within_threshold	1
staging.reassign_cross_region	1 (promoted)
staging.reassign_large_scale	1 (promoted)
maintenance.predictive_alert	1
maintenance.auto_schedule	1
maintenance.auto_schedule_high_cost	1 (promoted)
incident.severity_triage	1
incident.liability_assessment	3 — not eligible
remarketing.disposition_recommend	1
remarketing.disposition_execute	1 (promoted)
financing.covenant_early_warning	1
telematics.tamper_event_flag	3 — not eligible

> **Before this ships:** The four "promoted" rows used Fleet's existing production audit history as promotion evidence rather than a fresh 90-day observation window. Pull the actual historical approval/override rate for each and put it on record before deploying — see AUTONOMY_TIERING_ENGINEER_INSTRUCTIONS.md §2 for the full verification checklist. Nothing in this section is live on the server yet.

9. Deployment Procedure

```
cd infrastructure/terraform
terraform init
terraform plan  -var-file=environments/prod.tfvars
terraform apply -var-file=environments/prod.tfvars

cd ../kubernetes
python3 ../../scripts/generate_service_manifests.py
kubectl apply -k overlays/prod
```

Local development mirrors this topology at small scale via docker-compose.yml — one Postgres container per architecture layer, single-broker Kafka, single Redis, gateway as the only port exposed to the host.

10. New-Service Build Recipe

- Add the service to services-catalog.yaml: name, next reserved port, layer, service account, status: PLANNED.
- Add its database name to terraform/modules/cloudsql/variables.tf; run terraform apply to provision it.
- Run scripts/generate_service_manifests.py to generate its Kubernetes manifests.
- Scaffold the FastAPI service from the backend template; wire it to its Cloud SQL database; register its Kafka topics and RBAC permissions.

- If it needs a specialist agent, add that agent's actions to the appropriate tier in tier-policy.yaml — realtime if a human is waiting in-session, batch otherwise.
- Flip status to READY only once tests pass — this repository's documentation reflects reality, not intent.

11. Testing & Verification Discipline

- node --check syntax validation after every module addition.
- Python structural checks: balanced tags, no duplicate declarations, every handler resolves.
- jsdom runtime smoke tests covering edge cases, not just the happy path.
- For every Tier 1 autonomy action: a test asserting no approval card renders and the action executes and logs correctly.
- For every Tier 3 action: a test asserting the approval card renders and cannot proceed without an accountable-role selection.

12. Rollout & Promotion Plan

- Ship conservative for modules without production history — default to Tier 2, promote later with evidence.
- Collect 90 days of AI-004 audit data per action type: approval rate, override rate.
- Promote Tier 2 → Tier 1 only through the same evaluation-gate AI-003 uses for prompt changes — a reviewed config change, not a silent drift.
- Directed promotion (Fleet's path): for modules with existing production history, use the historical audit data as evidence before the code ships, rather than waiting 90 days going forward.
- Tier 3 is never promoted. That's a legal/compliance conversation, not an engineering one.

13. Related Documents

- docs/PORT_REGISTRY.md — authoritative port list
- docs/SERVICE_INVENTORY.md — per-service build status and notes
- docs/AI_ORCHESTRATION_SCALING.md — full rationale for the realtime/batch split
- docs/AUTONOMY_TIERING_ENGINEER_INSTRUCTIONS.md — the module-by-module autonomy rollout, including the complete Fleet Management diff
- VOYQI_PRD_v3.docx — the sole authoritative product requirements document
- VOYQI Enterprise Administration Screen Specification — 122-screen catalog across 17 primary panels